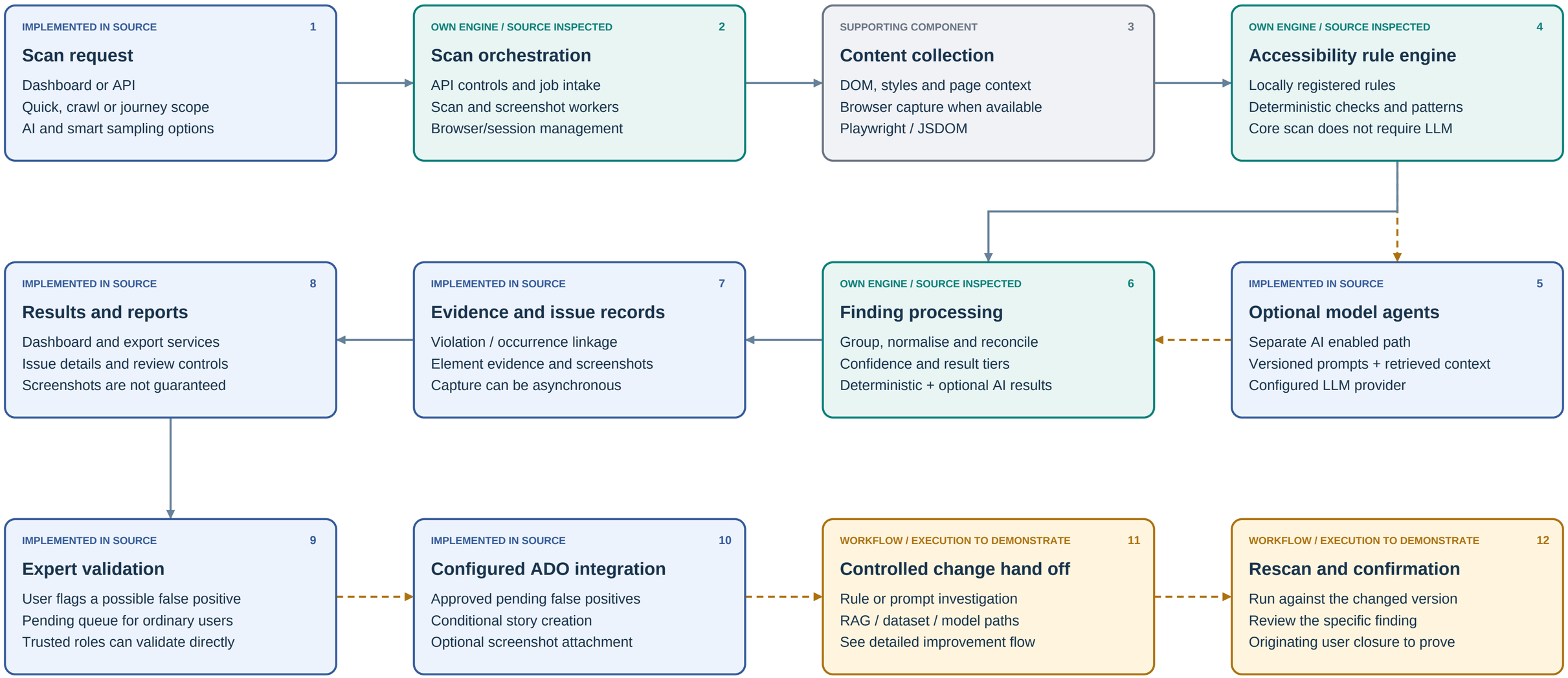
IntelliA11y | Scan and assurance architecture

Custom accessibility analysis with optional model interpretation and expert review



API stores. PostgreSQL / vector context and Redis. Separate CLI. Own rules with Puppeteer / JSDOM; outside this API workflow.

Own engine boundary. Primary API/CLI paths execute local rules. axe appears in a separate comparison script, not the inspected core scan path.

Ticket creation does not prove automatic code changes, fine tuning, deployment or final user confirmation.

Solid = inspected source path. Dashed = optional or human/external hand off. Code presence does not prove deployment or outcomes.

IntelliA11y | Expert governed improvement loop

End to end improvement process, with implemented entry points and explicit execution hand offs

IMPLEMENTED IN SOURCE 1
Scan and inspect
LLM enabled or disabled
Open a violation and evidence

IMPLEMENTED IN SOURCE 2
Report potential false positive
Originating user submits a label
Retain the issue and scan identity

IMPLEMENTED IN SOURCE 3
Expert decision
Pending review approval or rejection
Review outcome can differ from flag

IMPLEMENTED IN SOURCE 4
Create linked ADO story
Configured approved review path
Rule / agent prompt instructions

WORKFLOW / EXECUTION TO DEMONSTRATE 5
AI or engineer picks up work
Investigate the issue and evidence
Ticket consumer wiring to verify

WORKFLOW / EXECUTION TO DEMONSTRATE 6
Choose the corrective action
Rule logic, prompt or knowledge
Model change only when justified

WORKFLOW / EXECUTION TO DEMONSTRATE 7
Rule or prompt change
Review the source / prompt diff
Version and test the correction

WORKFLOW / EXECUTION TO DEMONSTRATE 8
HTML regression fixture
Expected positive and negative cases
Verify old and changed behaviour

IMPLEMENTED IN SOURCE 9
RAG knowledge update
Embed and retrieve approved content
Changes context, not model weights

IMPLEMENTED IN SOURCE 10
Curated dataset candidates
Exports exist; verify full lineage
Session to builder link incomplete

WORKFLOW / EXECUTION TO DEMONSTRATE 11
Fine tune and serve candidate
QLoRA + conversion scripts exist
Completed run / serving to prove

WORKFLOW / EXECUTION TO DEMONSTRATE 12
Evaluate candidate version
Independent holdout + HTML cases
Pass acceptance before promotion

WORKFLOW / EXECUTION TO DEMONSTRATE 13
Accept and record the change
Link accepted version and tests
No automatic closure established

WORKFLOW / EXECUTION TO DEMONSTRATE 14
Resolve / reassign ADO item
Record real work item transitions
Reassignment callback not evidenced

WORKFLOW / EXECUTION TO DEMONSTRATE 15
Return to originating user
Requested business process step
Reporter linkage must be shown

WORKFLOW / EXECUTION TO DEMONSTRATE 16
Fresh rescan and confirmation
Verify same case on new version
Confirm corrected result or reopen

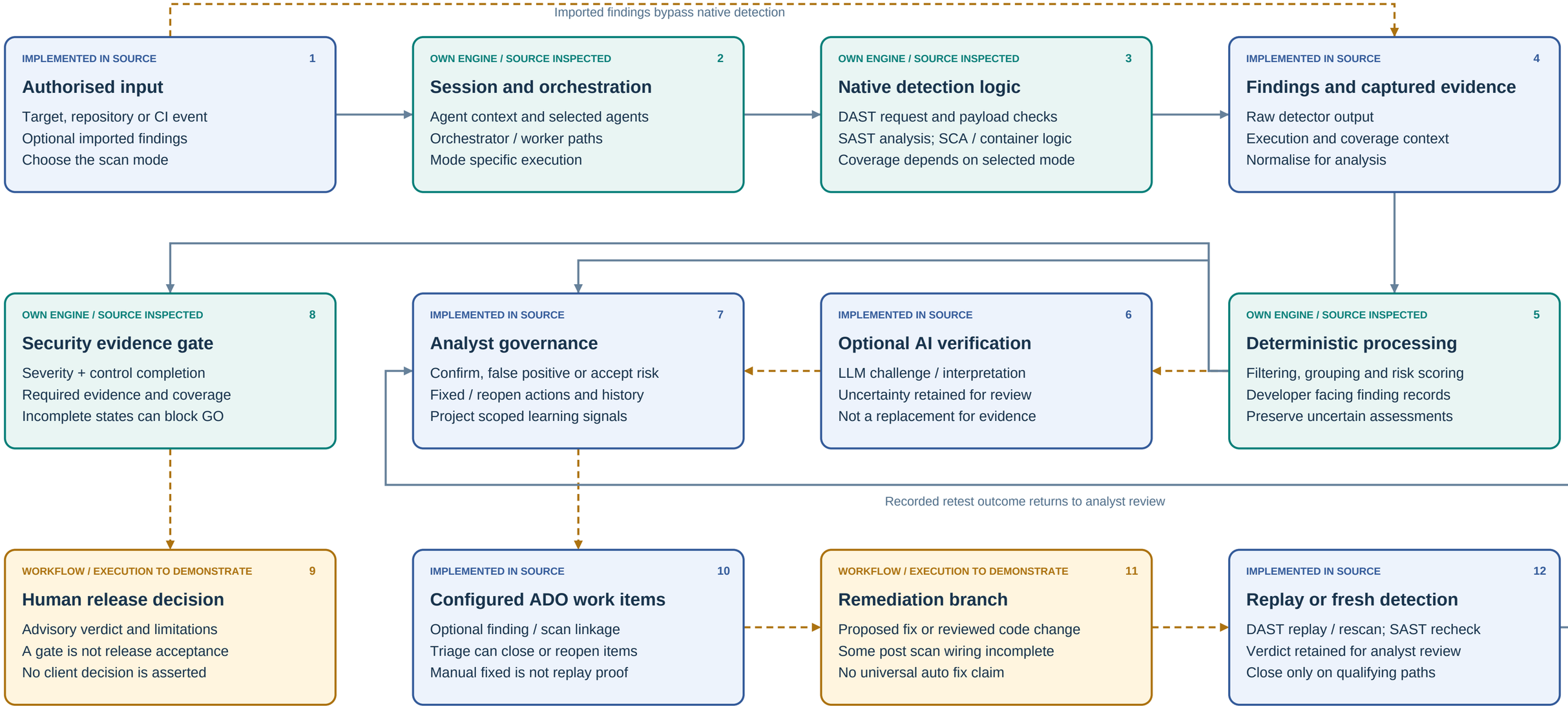
Rule edits, prompt edits, RAG updates and fine tuning are separate mechanisms. A false positive review need not trigger all four.

Solid links show inspected workflow connections. Dashed links show intended/manual or external execution; they are not claimed autonomous.

Dataset overlap must be resolved before independent uplift claims. A generated HTML fixture is a test case, not evidence of a client outcome.

IntelliSec | Native detection and governed remediation

Mode specific detector logic, evidence led triage and explicit retest branches



Improvement mechanisms. Scoped suppressions and rule candidates; prompt versions, A/B mechanisms and training data suggestions.

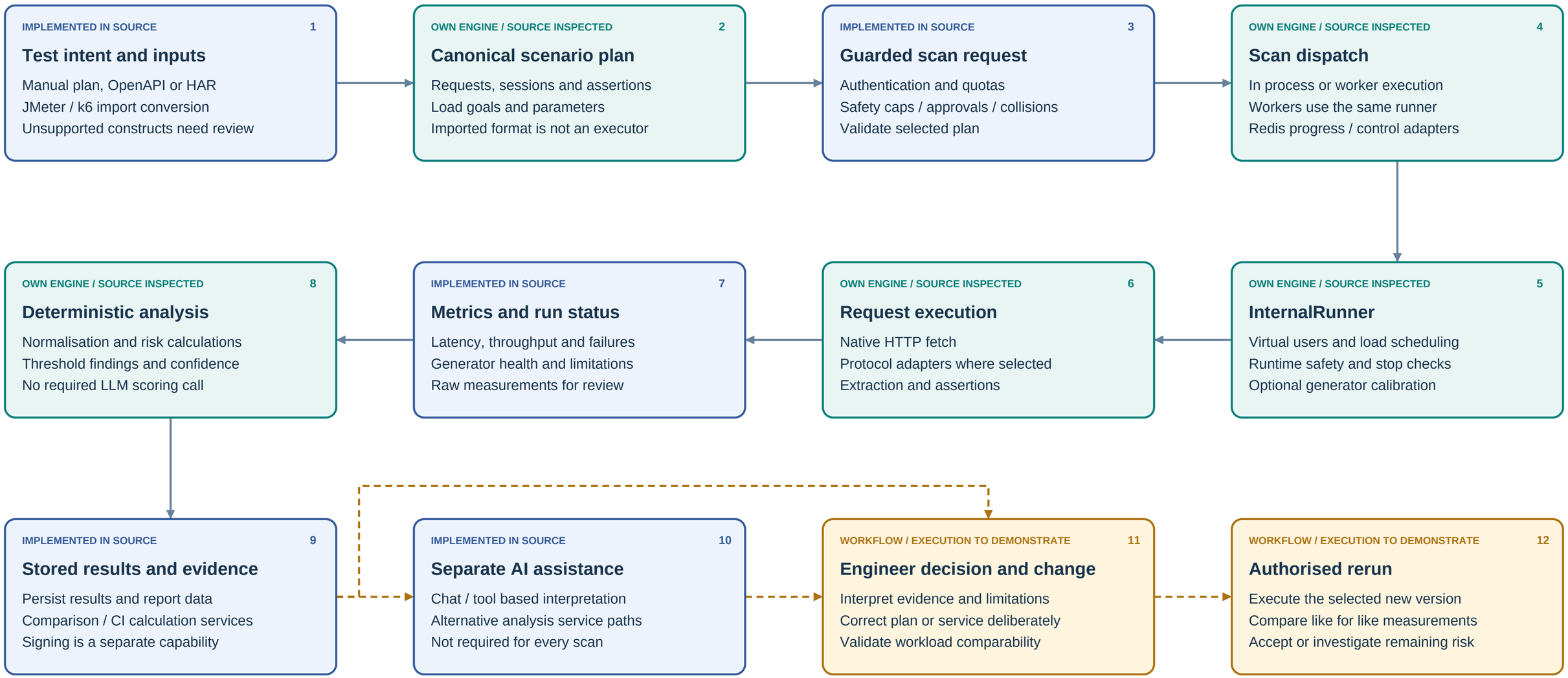
Own engine boundary. Native detectors coexist with optional imports, advisory feeds, policy tools and utility integrations. No zero dependency claim.

Retest backed and manual closure paths differ. Do not equate every fixed status with a verified fix.

Solid = inspected source path. Dashed = optional or human/external hand off. Code presence does not prove deployment or outcomes.

Intelliperf | Native workload and analysis architecture

Deterministic execution and measurement with separate AI assistance



Prototype boundary. Auto remediation actions and rollback are simulated. Mock regional runners are not proof of live distributed execution.

Own engine boundary. The inspected core invokes InternalRunner and native requests, not k6/JMeter executables. AI and imports are separate components.

Calibration can estimate capacity; scoring and comparison semantics need runtime validation. Separate from client PerfOps / Perf Insights.

Solid = inspected source path. Dashed = optional or human/external hand off. Code presence does not prove deployment or outcomes.